CryptoSim

Von Sara Kocak, Lucas Kleineidam, Jonathan Karrer und Julian Schüler

Softwareengineering | DHBW | Semester 3/4

Projektüberblick

Vision, Ziele & Funktionsumfang

Motivation

1

Innovatives
Krypto-Planspiel

2

Barrierefreier
Marktzugang

3

Praktische
Lernerfahrung

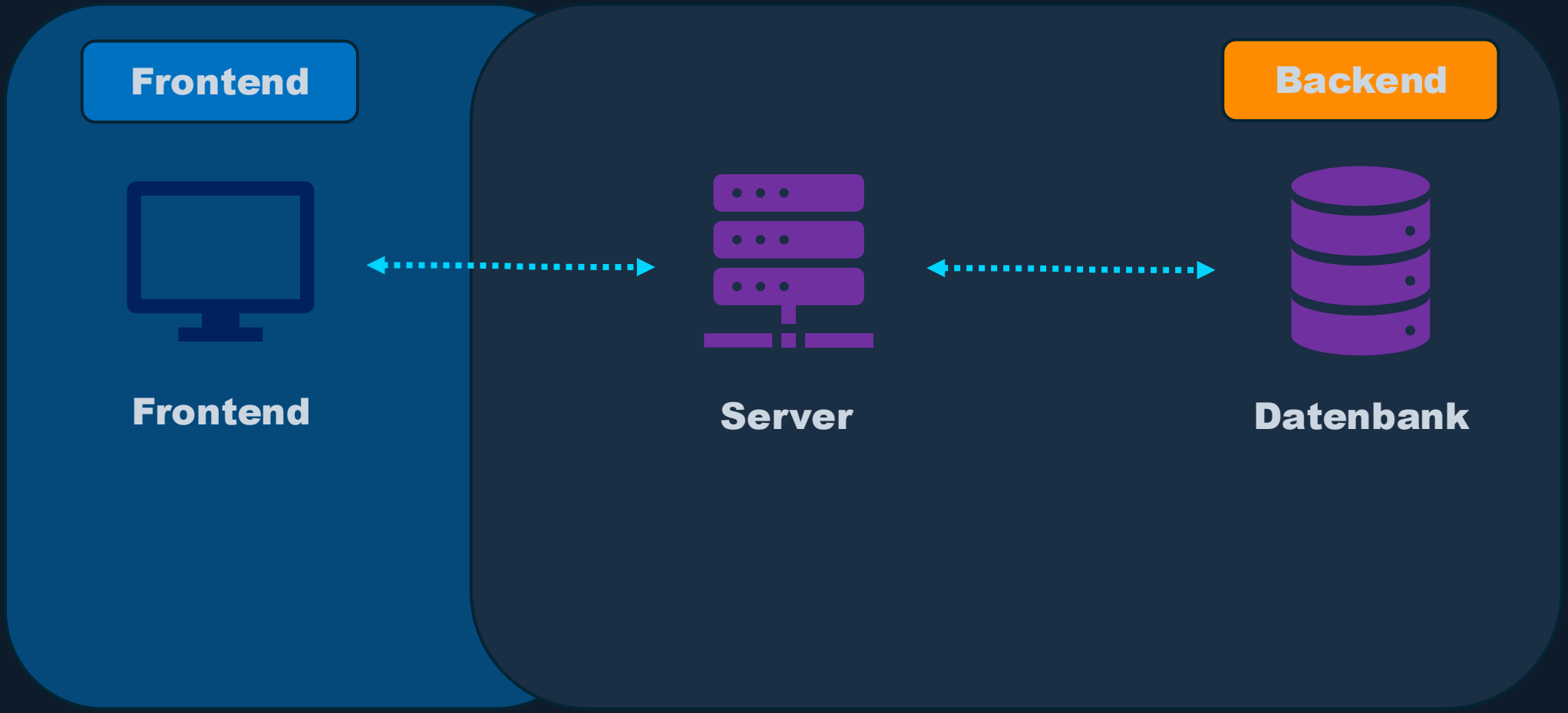
LIVE DEMO

HomeScreen des CryptoSim

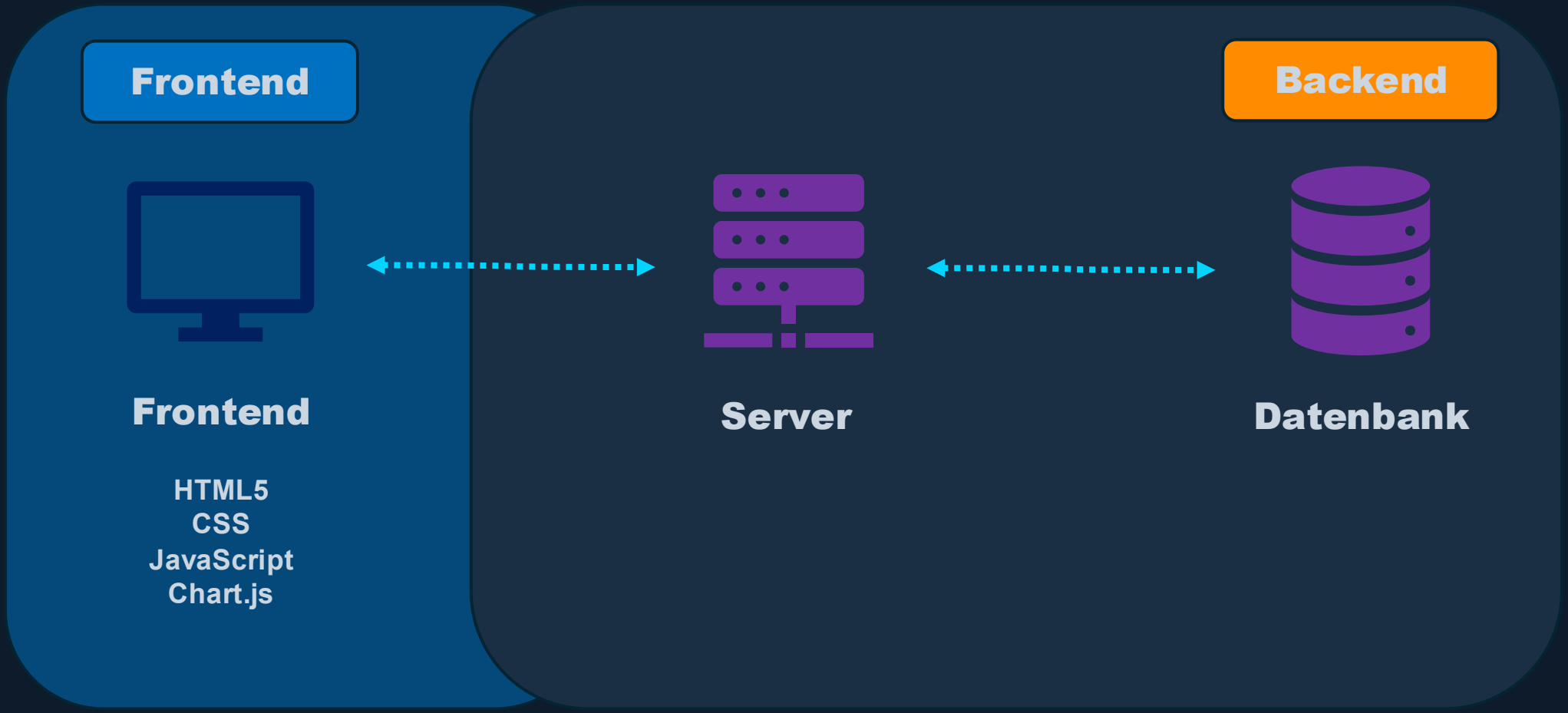
Technische Umsetzung

Tech Stack, CI/CD & Deployment

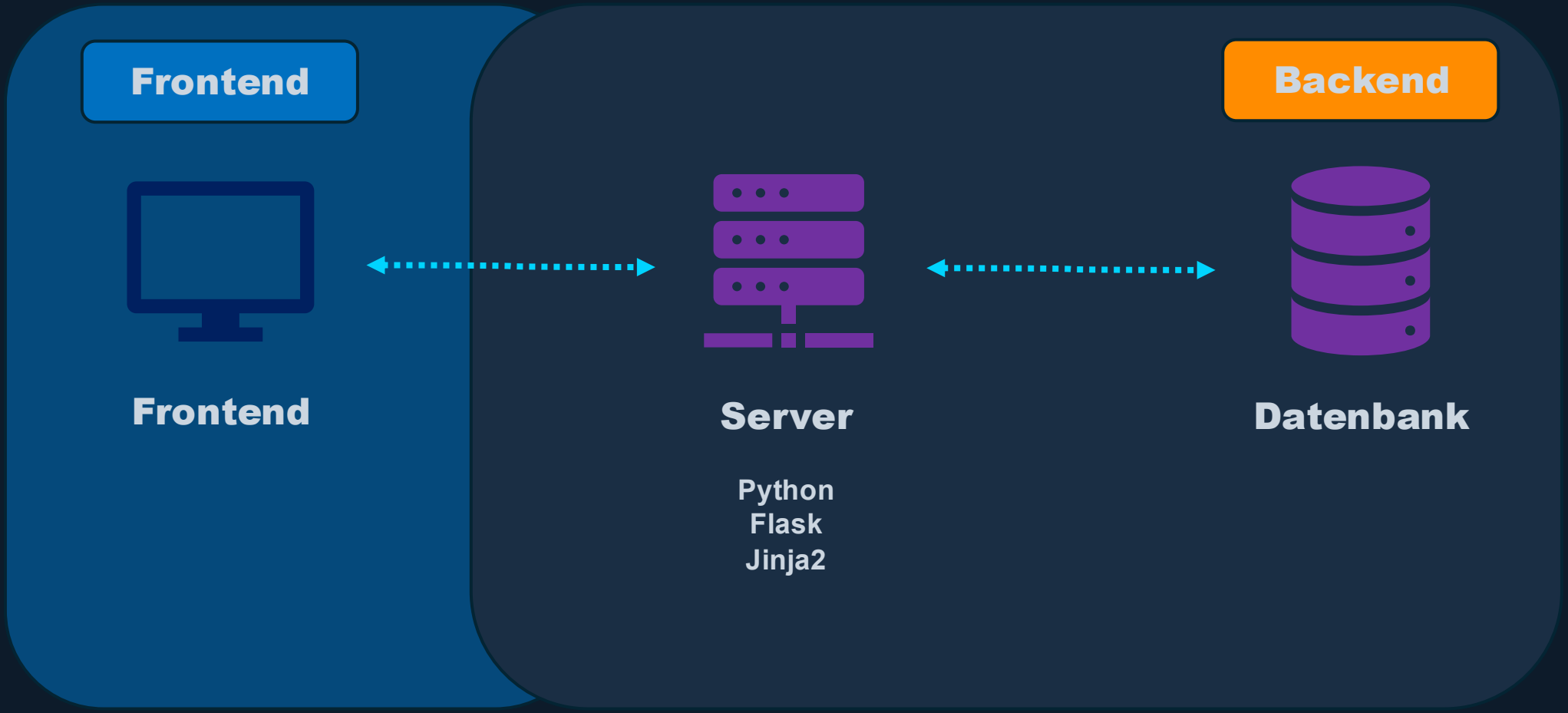
Tech Stack



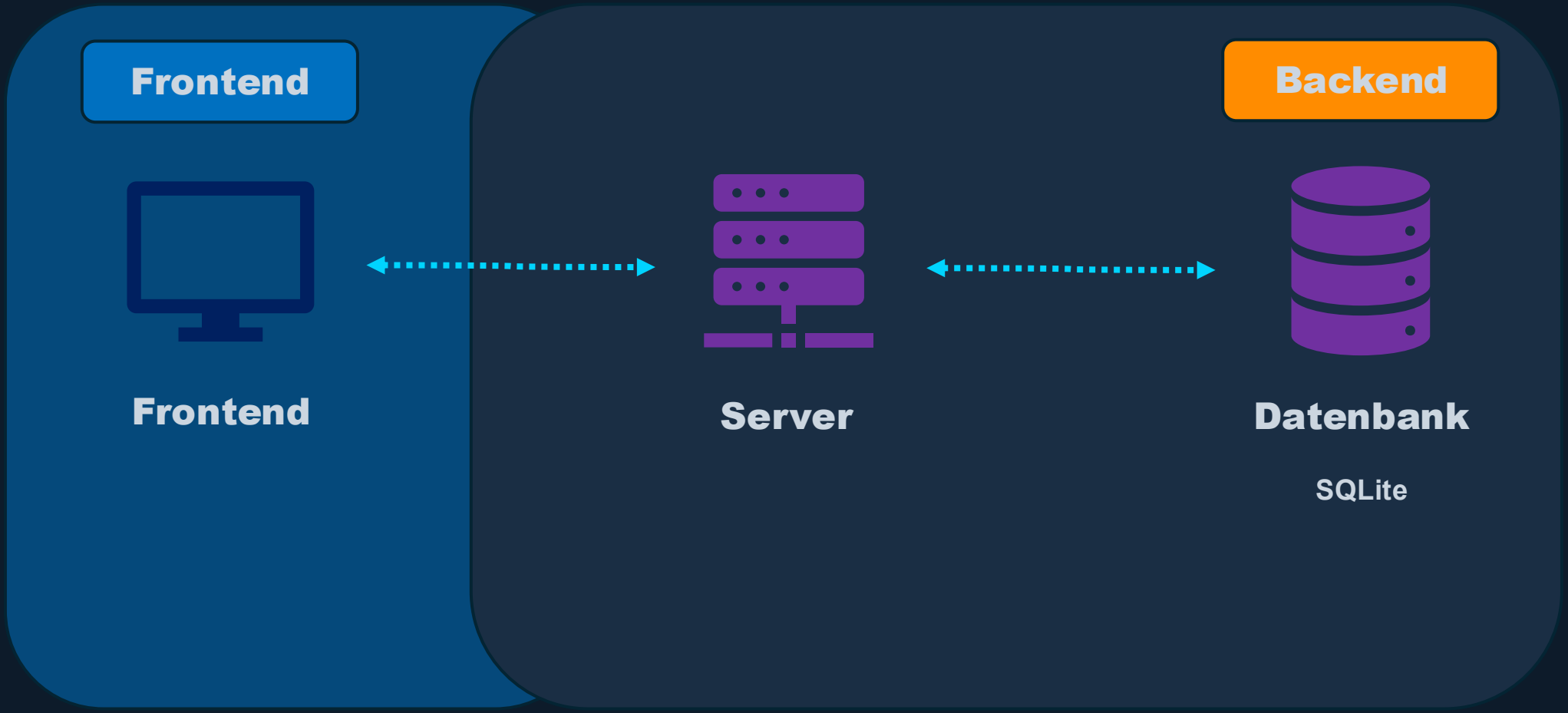
Tech Stack


Frontend
Backend
Frontend
HTML5
CSS
JavaScript
Chart.js
Server
Datenbank

Tech Stack


Frontend
Backend
Frontend
Server
Python
Flask
Jinja2
Datenbank

Tech Stack



CI Pipeline

CI Pipeline

- Repo wird auf der VM gepullt
- Python installieren
- Dependencies installieren
- Syntax checken
- Docker Image bauen
- Docker starten
- Erreichbarkeit des Cryptobroker testen
- Close/Remove Docker Container

Architektur

Systemdesign, Datenbank & Prinzipien

Architekturüberblick

Controller

app.py - Routen & Steuerung

Service-Layer

MarketService · PortfolioService

Model / Datenbank

models/ - Datenbanklogik

View

templates/ - HTML-Darstellung

- base.html
- dashboard.html
- portfolio.html
- transactions.html
- coin_detail.html
- home.html · profile.html

Konkrete Architekturentscheidungen

Zentrale Datenbank-Klasse

- Einzige Verbindungsquelle für alle Models
- Änderungen am DB-Pfad nur an einer Stelle
- database.py verwaltet sqlite3.connect()

Session-basierte Authentifizierung

- acc_id in der Flask-Session gespeichert
- Geschützte Routen prüfen Session vor Rendern
- Weiterleitung zu /profile falls nicht eingeloggt

Thin Controller - Fat Service

- Routen enthalten keine Geschäftslogik
- Gesamte Logik liegt im Service-Layer
- app.py ruft nur Service-Methoden auf

Datenbankaufbau

accounts

- id
- name
- pw
- created_at

coins

- id
- name
- symbol
- current_price
- market_cap
- market_cap_rank
- image (BLOB)

transactions

- id
- coin_id
- acc_id
- price
- amount
- type (BUY/SELL)
- timestamp

[coin]_history

- timestamp_ms
- price
- (je Coin eine eigene Tabelle)

Designprinzipien

Single Responsibility Principle (SRP)

Jede Datei hat genau eine Aufgabe.
Beispiel: database.py stellt nur die Verbindung her.

DRY (Don't Repeat Yourself)

base.html als zentrales Template.
Alle Seiten erben Navbar & Flash-Messages.

Separation of Concerns (SoC)

Datenhaltung, Logik & Darstellung klar getrennt.
Datenbank austauschbar ohne Template-Änderung.

Defensive Programming

Fehler werden aktiv abgefangen.
Überverkauf -> Fehlermeldung
Kein Bild -> Fallback-PNG.

Projektmanagement

Sprints, Zeitplanung & Qualität

Sprint-Zeitstrahl

Okt 2025 – Dez 2025 · Insgesamt 5 Sprints

Sprint 0 (Kickoff)

Okt 2025

Projektregeln, DoD, Design

Sprint 1 (14 Tage)

10.–23.11.

JavaFX-Grundlage, Navigations-GUI

Sprint 2 (7 Tage)

24.–30.11.

Technologie-wechsel zu Python/Flask

Sprint 3 (7 Tage)

01.–07.12.

CI/CD, Docker, Blog

Sprint 4 (7 Tage)

08.–14.12.

ASR, Blog, Doku

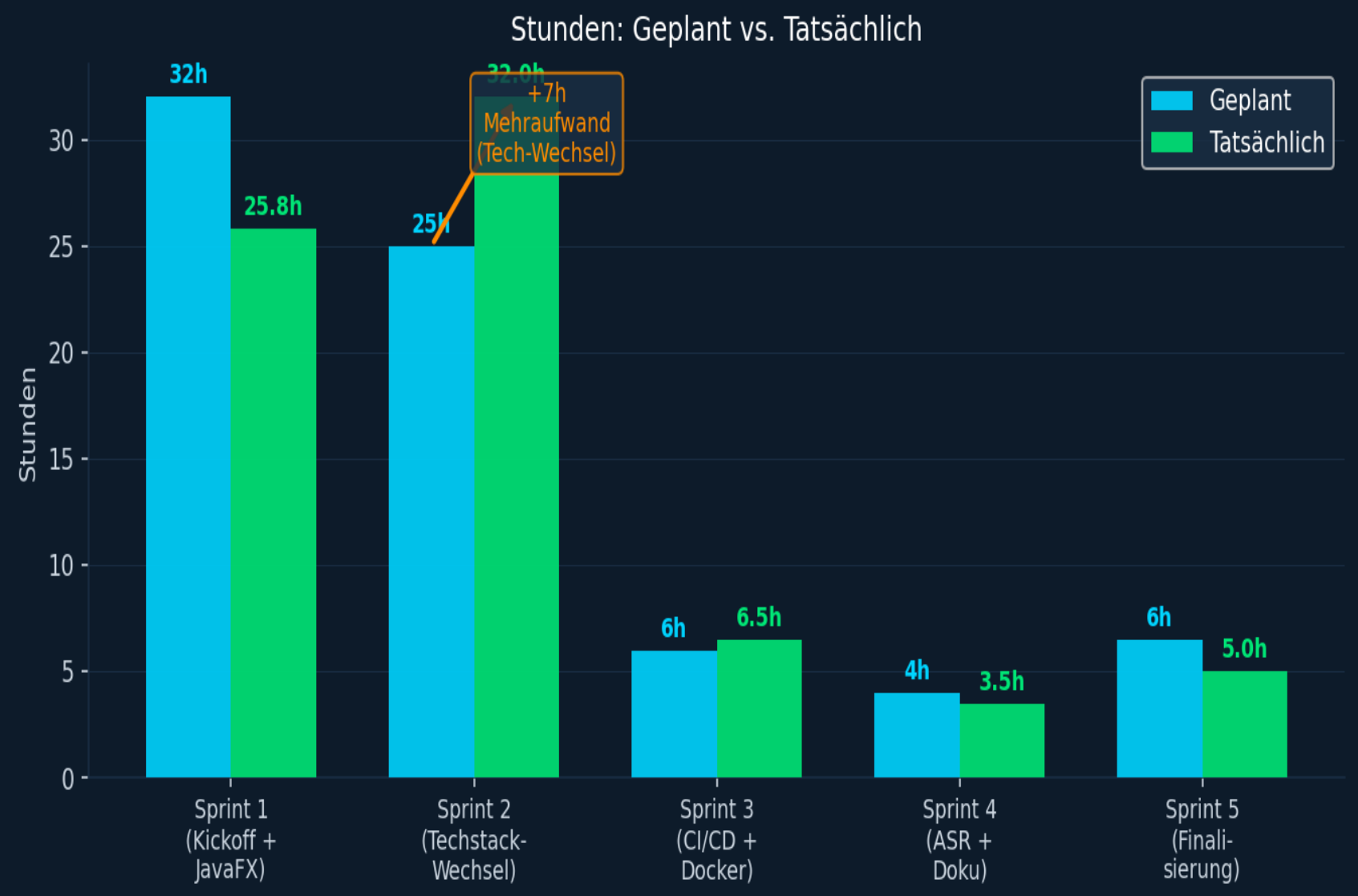
Sprint 5 (7 Tage)

15.–22.12.

SRS, Abschluss-blog, Finalisierung

Zeitplanung vs. tatsächlicher Aufwand

Geplante Stunden vs. tatsächliche Stunden pro Sprint

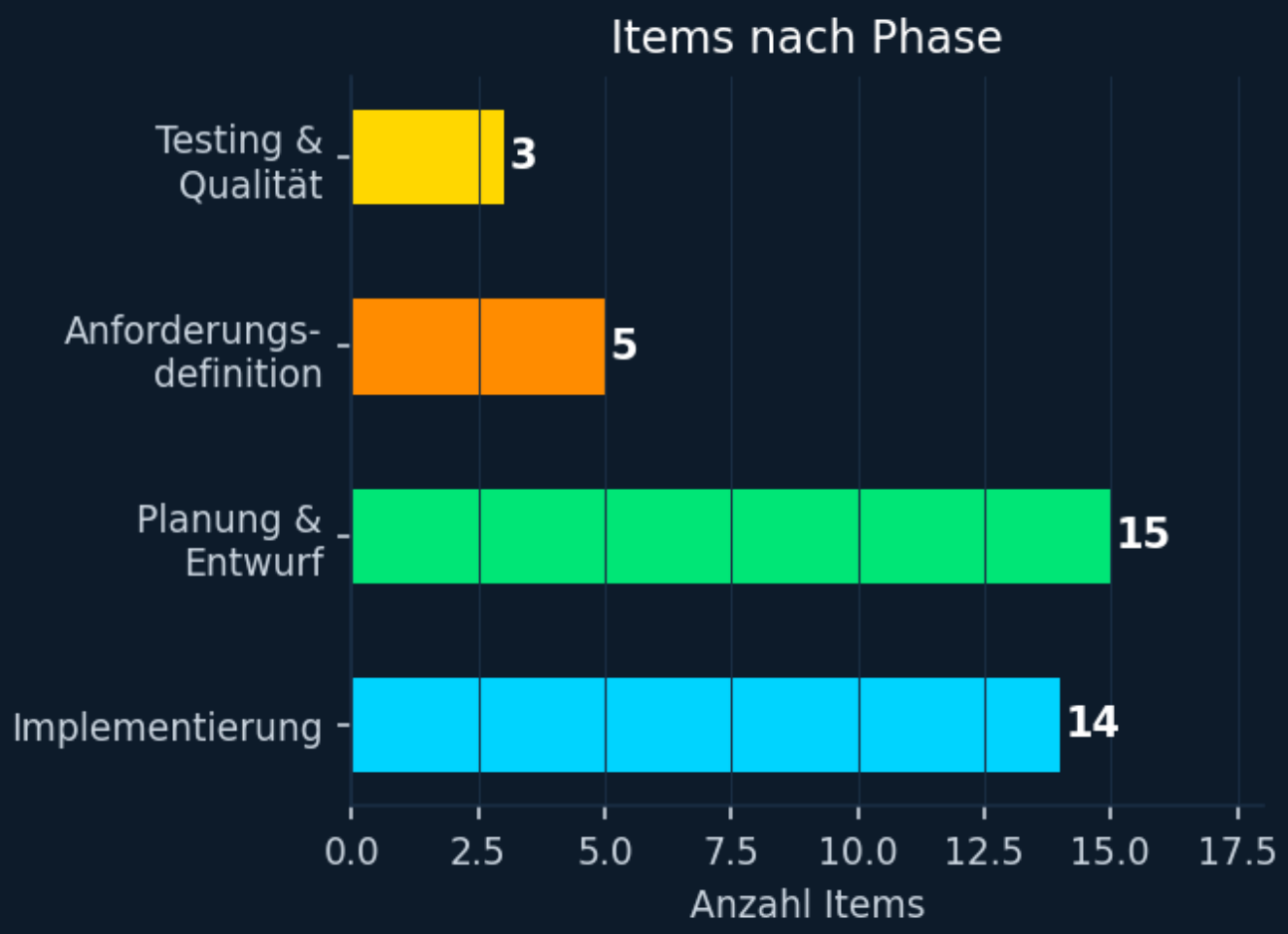


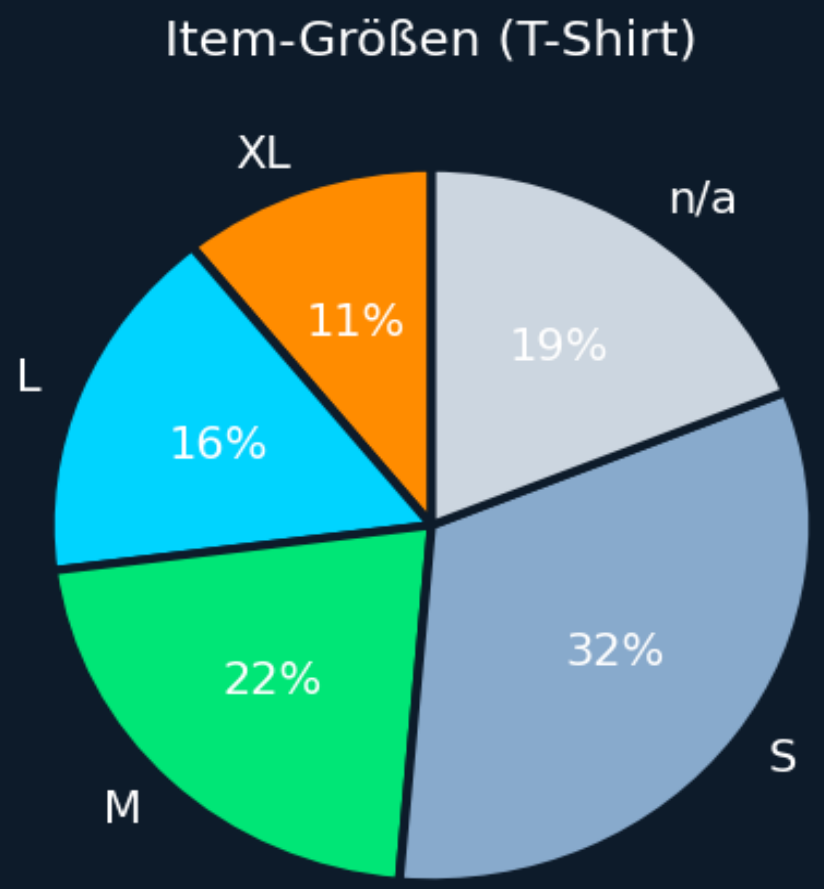
Erkenntnisse

- Sprint 1: ~6h gespart durch effizienten Design-Prozess
- Sprint 2: +7h durch Tech-Wechsel - bewusste Entscheidung
- Ab Sprint 3: Planung sehr präzise (Ø ±0.5h Abweichung)
- Gesamt Geplant: 73.5h
- Gesamt Tatsächlich: 72.8h (≈ ausgeglichen)
- Schätzgenauigkeit nahm über Sprints zu
- Größte Abweichung: Tech-Stack Wechsel Sprint 2

Issue-Statistiken & Projektstruktur

37 Items · 34 Done · 3 im Backlog · Phasen & Größen





Definition of Done & Qualitätsstandards

Verbindliche Akzeptanzkriterien für alle Arbeitspakete

Code & Implementierung

- Code muss vor dem Push von mind. 2 Devs geprüft werden

Testing

- Code muss compilieren
- Code muss Pipeline erfolgreich durchlaufen haben

Dokumentation

- Neu erstellter Code muss mindestens in der Datei dokumentiert werden
- Neue Builds benötigen Patch Notes / oder Blog Beitrag

Akzeptanzkriterien

- Dokumentation vorhanden
- Testing erfolgreich abgeschlossen
- Sprint eingetragen
- Tatsächlicher Zeitaufwand eingetragen

Lessons Learned – Gewonnene Erkenntnisse

Projektmanagement · Scrum-Prozess · Teamwork

Scrum & Agilität

- Kurze Sprint-Zyklen (7 Tage) ermöglichten schnelles Reagieren
- Tech-Stack-Wechsel in Sprint 2 war dank Scrum problemlos möglich

Zeitplanung

- Schätzgenauigkeit verbesserte sich deutlich über Sprints

Technologie & Architektur

- Frühe Technologie-Evaluation spart Aufwand langfristig
- JavaFX-zu-Flask-Wechsel: kurzfristig teuer, langfristig wertvoll

Teamarbeit & Prozess

- GitHub Projects mit Issues & Sprints: sehr gute Transparenz
- Pair-Review vor jedem Merge verbesserte Codequalität

Dokumentation

- Blog-Beiträge pro Sprint: gute Reflexion & Kommunikation

Ausblick – Nächste Schritte & Roadmap

Offene Backlog-Items · Weiterentwicklung · Vision

P0

Account-System

Nutzer können sich registrieren & einloggen
Personalisierte Portfolio-Daten

Issue #9 · Geplant: 5h · XL

P0

Live-Kursdaten

Automatische Markt-Updates

Issue #10 · Geplant: 7h · XL

P1

GUI-Überarbeitung

Issue #11 · Geplant: 20h · XL

P1

Fake-Guthaben & Trading

Virtuelles Startkapital für neue Nutzer
Kauf/Verkauf-Simulation

Issue #12 · Geplant: ~5h · M

P2

Watchlist / Favoriten

Nutzer können Lieblings-Coins speichern
Preisverlauf-Tracking

Issue #13 · Geplant: ~3h · S

Ziel

- Vollständige Trading-Simulation
- Live-Marktdaten
- Multi-User-Portfolio
- CI/CD Pipeline

Danke für eure Aufmerksamkeit

CryptoBroker · Software Engineering WS 2025/26

Lucas · Julian · Jonathan · Sara

Lucas-Klein1 · byJvlian · Khorrer · sxarv

Fragen?

github.com/Lucas-Klein1/CryptoBroker